

How Algorithms Learn to Trade: A Stochastic Optimal Control & Monte Carlo Exploration

Florent Ouabo Kamkumo

November 2025

Introduction and Motivation

Financial markets today are increasingly shaped by algorithms capable of making trading decisions in milliseconds. Whether the task is executing a large order, managing inventory risk, or reacting to intraday price dynamics, modern trading systems rely on quantitative models to determine *what to do next*.

The challenge lies in uncertainty: prices evolve randomly, trading costs are nonlinear, and future opportunities depend on present decisions. As a result, effective execution strategies must balance short-term revenue against long-term risk, adapt to volatility, and adjust as time runs down.

This naturally leads to the framework of **stochastic optimal control**. In this work, we illustrate how the combination of:

- a stochastic price process,
- an inventory state evolving with trades,
- a reward function capturing execution costs, and
- Monte Carlo-based dynamic programming,

produces an optimal, data-driven, adaptive trading policy. We also provide R implementations and animations to visualize how the optimal action map evolves over time.

1. Price Dynamics and Calibration of Volatility

We assume a discrete-time Gaussian random-walk model: $S_{t+1} = S_t + \sigma \varepsilon_{t+1}$, $\varepsilon_{t+1} \sim \mathcal{N}(0, 1)$.

Given historical prices $\{S_0, S_1, \dots, S_N\}$, define increments

$$\Delta S_t = S_t - S_{t-1}, \quad t = 1, \dots, N.$$

Under the model, $\Delta S_t \sim \mathcal{N}(0, \sigma^2)$, and thus a natural estimator of σ^2 is the sample variance:

$$\hat{\sigma}^2 = \frac{1}{N-1} \sum_{t=1}^N (\Delta S_t - \overline{\Delta S})^2, \quad \overline{\Delta S} = \frac{1}{N} \sum_{t=1}^N \Delta S_t.$$

In the sequel we use the calibrated volatility $\hat{\sigma}$ inside the dynamic programming algorithm.

2. Control Problem: Inventory and Rewards

The trader starts with inventory x_0 and must liquidate over T steps. At each time t , the action

$$a_t \in \mathcal{A}(x_t) := \{0, 1, \dots, \min(a_{\max}, x_t)\}$$

denotes the number of shares sold. Inventory evolves deterministically as:

$$x_{t+1} = x_t - a_t.$$

The one-step reward is

$$r_t(x_t, S_t, a_t) = a_t S_t - c a_t^2,$$

where $c > 0$ penalizes aggressive trading.

3. Objective Function and Value Function

A policy $\pi = (\pi_0, \dots, \pi_{T-1})$ consists of mappings $a_t = \pi_t(x_t, S_t)$. The objective function associated with π is

$$J^\pi(x_0, S_0) = \mathbb{E}^\pi \left[\sum_{t=0}^{T-1} r_t(x_t, S_t, a_t) \mid x_0, S_0 \right].$$

The optimal value function is

$$V_t(x, S) = \sup_{\pi} \mathbb{E}^\pi \left[\sum_{u=t}^{T-1} r_u(x_u, S_u, a_u) \mid x_t = x, S_t = S \right],$$

with terminal condition:

$$V_T(x, S) = 0.$$

In particular,

$$J^*(x_0, S_0) = V_0(x_0, S_0).$$

4. Bellman Equation

For $t = T - 1, \dots, 0$, the Bellman recursion is

$$V_t(x, S) = \max_{a \in \mathcal{A}(x)} \left\{ aS - ca^2 + \mathbb{E}[V_{t+1}(x - a, S + \sigma\varepsilon)] \right\}.$$

The only non-explicit term is the conditional expectation:

$$\mathbb{E}[V_{t+1}(x - a, S + \sigma\varepsilon)] = \int_{\mathbb{R}} V_{t+1}(x - a, S + \sigma z) \varphi(z) dz,$$

where φ is the standard normal density.

Because no closed form exists, we approximate this integral using Monte Carlo.

5. State-Space Discretization

We discretize the state space as follows:

- Inventory grid:

$$x_i \in \{0, 1, \dots, x_{\max}\}, \quad i = 0, \dots, I.$$

- Price grid:

$$S^{(j)} \in \{S^{(1)}, S^{(2)}, \dots, S^{(J)}\}, \quad j = 1, \dots, J.$$

For each t , we maintain a table

$$V_t(x_i, S^{(j)}), \quad i = 0, \dots, I, \quad j = 1, \dots, J.$$

The table V_{t+1} is fully known when computing V_t .

6. Monte Carlo Approximation of the Conditional Expectation

Fix a state $(x_i, S^{(j)})$ and a feasible action $a \in \mathcal{A}(x_i)$. Define the next inventory:

$$x' = x_i - a.$$

Simulate M i.i.d. shocks $z_m \sim \mathcal{N}(0, 1)$ and compute next-step prices:

$$S'_m = S^{(j)} + \sigma z_m.$$

To evaluate $V_{t+1}(x', S'_m)$ numerically, we interpolate the previously-computed grid values.

Nearest-Grid Approximation. Find j_m such that $S^{(j_m)}$ is closest to S'_m and set

$$V_{t+1}(x', S'_m) \approx V_{t+1}(x', S^{(j_m)}).$$

Linear Interpolation (preferred). Find j_m such that $S^{(j_m)} \leq S'_m \leq S^{(j_m+1)}$ and set

$$V_{t+1}(x', S'_m) \approx \lambda_m V_{t+1}(x', S^{(j_m)}) + (1 - \lambda_m) V_{t+1}(x', S^{(j_m+1)}),$$

where

$$\lambda_m = \frac{S^{(j_m+1)} - S'_m}{S^{(j_m+1)} - S^{(j_m)}}.$$

Monte Carlo estimator. The conditional expectation is approximated by:

$$\widehat{\mathbb{E}}_M[V_{t+1}(x - a, S + \sigma\varepsilon)] = \frac{1}{M} \sum_{m=1}^M V_{t+1}(x', S'_m).$$

7. Final Bellman Update

For each grid point $(x_i, S^{(j)})$:

$$V_t(x_i, S^{(j)}) \approx \max_{a \in \mathcal{A}(x_i)} \left\{ aS^{(j)} - ca^2 + \frac{1}{M} \sum_{m=1}^M V_{t+1}(x_i - a, S^{(j)} + \sigma z_m) \right\}.$$

All terms on the right-hand side are either:

- immediate rewards $aS^{(j)} - ca^2$ (explicit),
- or Monte Carlo interpolations of V_{t+1} (known from the previous DP step).

Thus, by backward induction starting from $V_T = 0$, we compute $V_{T-1}, V_{T-2}, \dots, V_0$, which provides the approximate optimal policy and optimal value function.

8. How to Compute the Grid Values $V_{t+1}(x_i, S^{(j)})$

We now explain in detail how the grid values

$$V_{t+1}(x_i, S^{(j)})$$

are computed in the dynamic programming algorithm.

8.1. Terminal Condition (Start of Backward Induction)

At time $t = T$ we define:

$$V_T(x, S) = 0 \quad \text{for all } (x, S).$$

Numerically:

$$V_T(x_i, S^{(j)}) = 0 \quad \text{for all } i, j.$$

This table serves as the base of the backward recursion.

8.2. Computing $V_{T-1}(x_i, S^{(j)})$ Explicitly

Using the Bellman equation at $t = T - 1$:

$$V_{T-1}(x, S) = \max_{a \in \mathcal{A}(x)} \{aS - ca^2 + \mathbb{E}[V_T(x - a, S + \sigma\varepsilon)]\}.$$

Since $V_T(\cdot, \cdot) = 0$, we have

$$\mathbb{E}[V_T(\cdot)] = 0,$$

hence

$$V_{T-1}(x, S) = \max_{a \in \mathcal{A}(x)} \{aS - ca^2\}.$$

Thus on the grid:

$$V_{T-1}(x_i, S^{(j)}) = \max_{a \in \mathcal{A}(x_i)} \{a S^{(j)} - ca^2\}.$$

This is a finite maximization over the discrete action set

$$\mathcal{A}(x_i) = \{0, 1, \dots, \min(a_{\max}, x_i)\},$$

and requires no Monte Carlo.

After this step, we have all values

$$V_{T-1}(x_i, S^{(j)}).$$

8.3. General Step: Computing $V_t(x_i, S^{(j)})$ for $t \leq T - 2$

Assume that all values $V_{t+1}(x_i, S^{(j)})$ have already been computed.

The Bellman equation at time t is:

$$V_t(x, S) = \max_{a \in \mathcal{A}(x)} \{aS - ca^2 + \mathbb{E}[V_{t+1}(x - a, S + \sigma\varepsilon)]\}.$$

8.3.1 Monte Carlo Approximation

Fix a grid point $(x_i, S^{(j)})$ and an action $a \in \mathcal{A}(x_i)$.

1. Compute next inventory:

$$x' = x_i - a.$$

2. Simulate M shocks $z_1, \dots, z_M \sim \mathcal{N}(0, 1)$.

3. For each m , compute the next simulated price:

$$S'_m = S^{(j)} + \sigma z_m.$$

4. Approximate the conditional expectation:

$$\widehat{\mathbb{E}}_M[V_{t+1}(x', S^{(j)} + \sigma \varepsilon)] = \frac{1}{M} \sum_{m=1}^M V_{t+1}(x', S'_m).$$

The key remaining task is to compute $V_{t+1}(x', S'_m)$.

8.4. Evaluating $V_{t+1}(x', S'_m)$ Using Grid Values

At time $t + 1$, the values

$$V_{t+1}(x_i, S^{(j)})$$

are already known and stored in a table.

However, the simulated price S'_m will typically not coincide with a grid point $S^{(j)}$.

There are two standard approaches:

Option A: Nearest Grid Point. Let j_m be such that $S^{(j_m)}$ is closest to S'_m . Then approximate:

$$V_{t+1}(x', S'_m) \approx V_{t+1}(x', S^{(j_m)}).$$

Option B: Linear Interpolation (Preferred). Find j_m such that

$$S^{(j_m)} \leq S'_m \leq S^{(j_m+1)}.$$

Define

$$\lambda_m = \frac{S^{(j_m+1)} - S'_m}{S^{(j_m+1)} - S^{(j_m)}}, \quad \lambda_m \in [0, 1].$$

Then approximate:

$$V_{t+1}(x', S'_m) \approx \lambda_m V_{t+1}(x', S^{(j_m)}) + (1 - \lambda_m) V_{t+1}(x', S^{(j_m+1)}).$$

Both values on the right-hand side are known from the stored table $V_{t+1}(x_i, S^{(j)})$.

8.5. Final Expression for the Grid Value $V_t(x_i, S^{(j)})$

Putting all steps together, for each grid point $(x_i, S^{(j)})$:

$$V_t(x_i, S^{(j)}) \approx \max_{a \in \mathcal{A}(x_i)} \left\{ a S^{(j)} - ca^2 + \frac{1}{M} \sum_{m=1}^M V_{t+1}(x_i - a, S^{(j)} + \sigma z_m) \right\}.$$

Each term

$$V_{t+1}(x_i - a, S^{(j)} + \sigma z_m)$$

is evaluated using either nearest-grid lookup or interpolation.

Summary

- Compute V_T first (all zeros).
- Compute V_{T-1} explicitly (no Monte Carlo needed).
- For $t = T - 2, \dots, 0$, compute each value $V_t(x_i, S^{(j)})$ by:
 - simulating Monte Carlo price increments,
 - evaluating V_{t+1} at simulated next states via interpolation,
 - maximizing over the action set.
- At each step, V_{t+1} is just a stored numerical table.

Numerical Illustration

To demonstrate how the optimal execution algorithm behaves under different market conditions, we simulate three independent price trajectories and apply the time-dependent optimal policy computed by the dynamic-programming procedure. For each scenario, we display:

- the simulated price path,
- the corresponding inventory evolution,
- the sequence of optimal actions,
- the realized rewards,
- and an animation illustrating the full evolution in time.

The static figures provide a snapshot of the key variables, show the dynamic behaviour step by step.

Remark on Units and Time Scale. The numerical illustrations presented below are generic and can be interpreted under different market conventions. The time index $t = 0, 1, \dots, T$ may represent seconds (high-frequency execution), minutes (intraday execution), or longer intervals depending on the application. Likewise, the price variable S_t is expressed in, any monetary unit (dollars,..) may be used. The inventory variable x_t corresponds to the quantity of shares (or lots) remaining to be executed; one unit of x_t may represent 1 share, 10 shares, 10^2 shares, or even 10^3 shares depending on the desired granularity. The model is dimension-free: only the relative scaling matters, and the optimal policy adapts accordingly once the units are fixed.

Scenario 1: Rising Market with Early Liquidation Opportunities

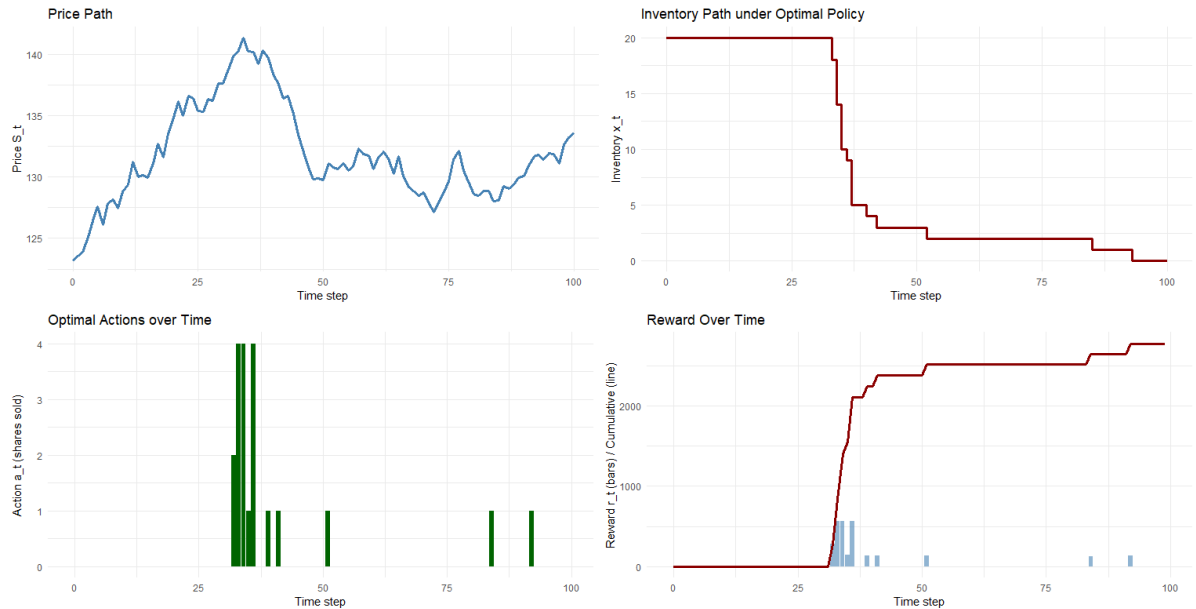


Figure 1: Price increases early, encouraging the policy to sell aggressively in the first third of the horizon. Inventory drops quickly and cumulative rewards rise sharply once high prices are exploited.

Scenario 2: Gradual Uptrend with Late Execution

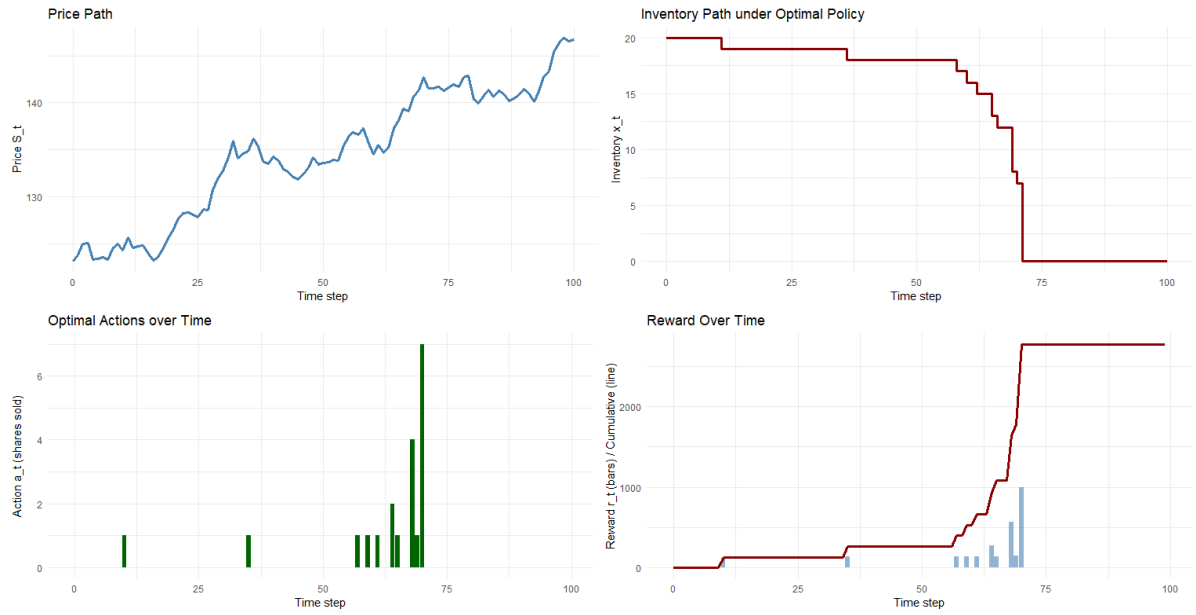


Figure 2: A smoother upward trend leads to slower execution: the algorithm waits for better prices and liquidates the position later in the horizon, when prices reach a local maximum.

Scenario 3: Declining Market with Final Rush

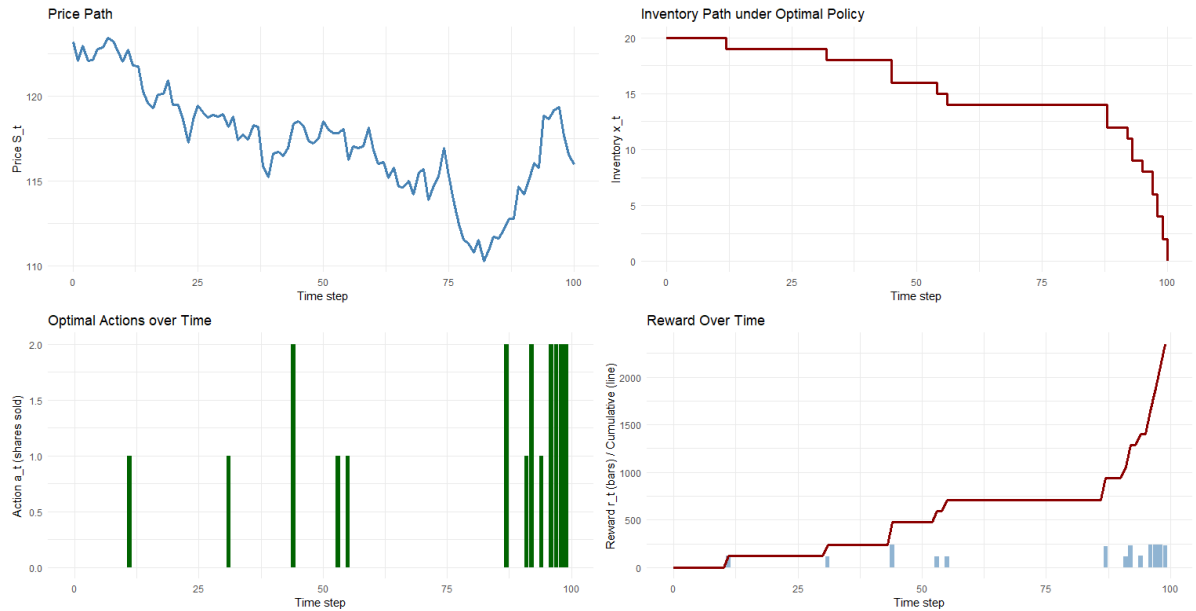


Figure 3: In a downward environment, the policy postpones heavy selling until prices slightly recover, followed by a sharp and unavoidable liquidation near the horizon to avoid ending with leftover inventory.

9. Conclusion

This work demonstrates how stochastic optimal control, combined with Monte Carlo simulation, can be used to design adaptive and data-driven execution strategies. The method provides a transparent and flexible alternative to black-box reinforcement learning models, while capturing the essential uncertainty of financial markets.

The complete implementation, including calibration routines, dynamic programming solver, simulation engine, and animation scripts, is available on my GitHub repository.

10. Extension : Next Post on this Subject

An important extension of this work is to calibrate the price dynamics directly from real stock data and to execute the optimal policy on the resulting model. For example, using historical daily or intraday prices of a liquid asset such as AAPL or MSFT, one may estimate the empirical volatility σ , drift μ , and distribution of short-term returns. These parameters immediately update the stochastic price transition

$$S_{t+1} = S_t + \mu \Delta t + \sigma \sqrt{\Delta t} \varepsilon_{t+1},$$

so that the dynamic program operates on a model that reflects the actual behaviour of the chosen stock.

Once the model is calibrated, the backward dynamic-programming step produces a new set of optimal policy surfaces $A_t(x, S)$ tailored to the volatility environment of that specific asset. Running a forward simulation with the optimal policy then reveals how the algorithm reacts when real-market features are present: higher volatility typically leads to more conservative early trading and a steeper liquidation near the end of the horizon, while more stable price dynamics allow smoother execution over time. In all cases, the optimal strategy automatically adapts to the empirical risk profile of the underlying stock.

References

- [1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [2] Á. Cartea, S. Jaimungal, and J. Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.